

Guide de Développement - JobbingTrack

[← Retour au README principal](#)

Vue d'ensemble

Ce guide complet couvre tout ce dont vous avez besoin pour développer et contribuer au projet JobbingTrack, un système de suivi de candidatures avec architecture microservices moderne.

Table des matières

-  [Configuration de l'environnement](#)
 -  [Architecture du projet](#)
 -  [Développement](#)
 -  [Tests](#)
 -  [Outils de développement](#)
 -  [Monitoring et debugging](#)
 -  [Déploiement](#)
 -  [Contribution](#)
 -  [Résolution de problèmes](#)
-

Configuration de l'environnement

Prérequis système

Logiciels requis

- **Docker** 20.10+ et **Docker Compose** 2.0+
- **Node.js** 20.x LTS (pour le développement local)
- **Git** 2.30+
- **Make** (pour les commandes automatisées)
- **PostgreSQL** 15+ (optionnel, pour le développement local)
- **Redis** 7+ (optionnel, pour le développement local)

Ressources recommandées

- **RAM** : 8GB minimum, 16GB recommandé
- **CPU** : 4 cœurs minimum
- **Stockage** : 10GB d'espace libre
- **Réseau** : Connexion internet stable

Installation et configuration

1. Clonage du repository

```
# Cloner le projet
git clone https://github.com/PavelDelhomme/JobbingTrack.git
cd JobbingTrack

# Vérifier la structure
ls -la
tree -L 2 # Si tree est installé
```

2. Configuration des variables d'environnement

```
# Copier le template d'environnement
cp .env.example .env

# Éditer le fichier .env
nano .env # ou code .env
```

Contenu du fichier `.env` :

```
# =====
# CONFIGURATION DE BASE
# =====

# Environnement
NODE_ENV=development
COMPOSE_PROJECT_NAME=jobbingtrack

# =====
# BASE DE DONNÉES
# =====

# PostgreSQL
POSTGRES_DB=jobbingtrack
POSTGRES_USER=jobbingtrack
POSTGRES_PASSWORD=jobbingtrack123
DATABASE_URL=postgresql://jobbingtrack:jobbingtrack123@postgres:5432/jobbingtrack?

# =====
# CACHE ET SESSIONS
# =====

# Redis
```

```
REDIS_URL=redis://redis:6379
```

```
# =====
# AUTHENTICATION
# =====
```

```
# JWT
JWT_SECRET=your-secret-key-change-in-production-2025
JWT_REFRESH_SECRET=your-refresh-secret-change-too-2025
JWT_EXPIRES_IN=1h
JWT_REFRESH_EXPIRES_IN=7d
```

```
# =====
# EMAIL ET NOTIFICATIONS
# =====
```

```
# SMTP (optionnel)
SMTP_HOST=smtp.ovh.net
SMTP_PORT=587
SMTP_USER=candidatures@delhomme.ovh
SMTP_PASS=V**Uw61^3*bz5c2AFrx&2d&%
SMTP_FROM=JobbingTrack <candidatures@delhomme.ovh>
```

```
# =====
# FRONTEND
# =====
```

```
# URLs
FRONTEND_URL=http://localhost:8080
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8080,http://localhost:5173
```

```
# =====
# LOGGING
# =====
```

```
LOG_LEVEL=info
LOG_FORMAT=json
LOG_TIMESTAMP=true
```

```
# =====
# SÉCURITÉ
# =====
```

```
# WAF (Web Application Firewall)
WAF_ENABLED=false
```

```
# Rate limiting
RATE_LIMIT_WINDOW=15
RATE_LIMIT_MAX=100
```

3. Premier démarrage

```
# Vérifier que Docker fonctionne
docker --version
docker-compose --version

# Construire et démarrer les services essentiels
make up

# Vérifier que tout fonctionne
make health

# Voir les logs
make logs
```

4. Accès aux services

```
# Frontend (interface web)
open http://localhost:8080

# API Gateway
open http://localhost:3000/health

# Monitoring cAdvisor
open http://localhost:8081

# Metrics Aggregator
open http://localhost:3014

# Prometheus (si activé)
open http://localhost:9090
```

Configuration avancée

Développement avec services externes

```
# Utiliser une base de données externe
DATABASE_URL=postgresql://user:pass@localhost:5432/jobbingtrack_dev

# Utiliser Redis externe
REDIS_URL=redis://localhost:6379
```

```
# Configuration pour le développement mobile
FLUTTER_EMULATOR=true
```

Configuration IDE

```
# Visual Studio Code
code .

# Extensions recommandées :
# - Docker
# - Node.js
# - TypeScript
# - Prettier
# - ESLint
# - GitLens
```

Configuration Git

```
# Configurer Git (si pas déjà fait)
git config user.name "Votre Nom"
git config user.email "votre.email@example.com"

# Vérifier la configuration
git config --list
```



Architecture du projet

Structure complète du projet

```
JobbingTrack/
├── 📁 backend/
│   ├── 🔗 api-gateway/
│   ├── 🔒 auth-service/
│   ├── 📋 application-service/
│   ├── 🏢 company-service/
│   ├── 👤 contact-service/
│   ├── 🎤 interview-service/
│   ├── ☎️ call-service/
│   ├── 📅 event-service/
│
# Services backend (15+ microservices)
# Point d'entrée API (Port: 3000)
# Authentification JWT (Port: 3001)
# Gestion des candidatures (Port: 3002)
# Gestion des entreprises (Port: 3003)
# Gestion des contacts (Port: 3004)
# Gestion des entretiens (Port: 3005)
# Gestion des appels (Port: 3006)
# Gestion des événements (Port: 3007)
```

			followup-service/	# Suivi des candidatures (Port: 3008)
			profile-service/	# Profils utilisateurs (Port: 3009)
			notification-service/	# Notifications (Port: 3010)
			workflow-service/	# Workflows automatisés (Port: 3011)
			dashboard-service/	# Analytics et dashboard (Port: 3007)
			security-service/	# Sécurité et audit (Port: 3017)
			system-metrics-service/	# Métriques système (Port: 3018)
			deployment-service/	# Gestion des déploiements (Port: 3016)
			docker-stats-service/	# Statistiques Docker (Port: 3015)
			metrics-aggregator-service/	# Agrégation des métriques (Port: 3014)
			monitoring/	# Configuration monitoring
			prisma/	# Schémas de base de données
			docker-compose.yml	# Configuration Docker
			README.md	# Documentation backend
			frontend/	# Application Next.js (Port: 8080)
			next.config.js	# Configuration Next.js
			src/	# Code source React/TypeScript
			tests/	# Tests frontend
			Dockerfile	# Container frontend
			package.json	# Dépendances
			README-IMPORT-FIX.md	# Documentation
			flutter-mobile-app/	# Application mobile Flutter (Port: 8090)
			lib/	# Code source Dart
			integration_test/	# Tests d'intégration
			Dockerfile	# Container Flutter
			pubspec.yaml	# Dépendances Flutter
			README.md	# Documentation mobile
			docker-compose.yml	# Configuration Docker principale
			docker-compose.prod.yml	# Configuration production
			docker-compose.test.yml	# Configuration tests
			Makefile	# Automatisation des tâches
			make.sh	# Script Make principal
			docs/	# Documentation complète
			ARCHITECTURE.md	# Architecture technique
			API.md	# Documentation API
			SERVICES.md	# Services disponibles
			DEVELOPMENT.md	# Guide développement
			TESTING-GUIDE.md	# Tests et qualité
			TROUBLESHOOTING.md	# Dépannage
			deployment-guide.md	# Déploiement
			security-guide.md	# Sécurité

	└─ 📖 README.md	# Index documentation
	└─ 🖋️ tests/	# Suite de tests complète
	└─ 🖋️ backend/	# Tests backend
	└─ 🖋️ frontend/	# Tests frontend
	└─ 🖋️ e2e/	# Tests end-to-end
	└─ 📊 reports/	# Rapports de tests
	└─ 📖 README.md	# Guide des tests
	└─ 📄 scripts/	# Scripts utilitaires
	└─ 🚀 setup/	# Scripts d'installation
	└─ 🐳 docker/	# Scripts Docker
	└─ 💾 database/	# Scripts base de données
	└─ 🔍 health/	# Scripts de santé
	└─ 📖 README.md	# Documentation scripts
	└─ ⚙️ config/	# Configuration
	└─ 📊 services.json	# Configuration services
	└─ 👤 profiles.json	# Profils Docker
	└─ 📦 package.json	# Dépendances root
	└─ 📦 package-lock.json	# Lock des dépendances
	└─ 📝 .env.example	# Template d'environnement
	└─ 📝 .gitignore	# Fichiers ignorés par Git
	└─ 📖 README.md	# README principal
	└─ 📝 LICENSE	# Licence du projet

Architecture des microservices

Structure standard d'un service backend

```

backend/service-name/
├── 🐳 Dockerfile # Configuration Docker
├── 📦 package.json # Dépendances Node.js
├── 🖋️ jest.config.js # Configuration Jest
├── 📖 README.md # Documentation du service
|
├── 💾 prisma/ # Base de données
|   ├── 📄 schema.prisma # Schéma Prisma
|   ├── 📁 migrations/ # Migrations
|   └── 📄 seed.js # Données de test
|
├── 📁 src/ # Code source
|   └── 🚀 server.js # Point d'entrée

```

```

|   └─ ⚙️ config/                # Configuration
|   └─ 🎮 controllers/          # Contrôleurs API
|   └─ 🛣️ routes/               # Routes API
|   └─ 🔧 services/             # Logique métier
|   └─ 🛡️ middlewares/          # Middlewares Express
|   └─ 🛒 utils/                # Utilitaires
|
└─ 🧪 tests/                   # Tests du service
    └─ 🎮 controllers/         # Tests des contrôleurs
    └─ 🔧 services/            # Tests des services
    └─ 🚀 server.test.js       # Tests d'intégration

```

Technologies utilisées

- **Backend** : Node.js 20+, Express.js, Prisma ORM
- **Frontend** : Next.js 14+, React 18+, TypeScript
- **Mobile** : Flutter (Dart), Provider state management
- **Base de données** : PostgreSQL 15+, Redis 7+
- **Container** : Docker 20.10+, Docker Compose 2.0+
- **Monitoring** : Prometheus, Grafana, cAdvisor
- **Tests** : Jest, Playwright, Supertest

Base de données et schémas

Architecture des données

- **Base unique** : PostgreSQL centralisée
- **Schémas séparés** : Un schéma par service
- **Migrations** : Prisma pour la gestion des schémas
- **Seed** : Données de test automatiques

Exemple de schéma Prisma

```

// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id      String  @id @default(uuid())
  email   String  @unique

```



```

name      String?
role      Role      @default(USER)
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

// Relations
applications Application[]
interviews Interview[]

@@map("users")
}

model Application {
  id          String      @id @default(uuid())
  userId      String
  companyId   String?
  position    String
  status      String      @default("applied")
  description String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  // Relations
  user        User        @relation(fields: [userId], references: [id])
  company     Company?    @relation(fields: [companyId], references: [id])

  @@map("applications")
}

```

Consultez la [documentation d'architecture complète](#) pour plus de détails techniques.

Développement

Démarrage rapide

Services essentiels

```

# Démarrer seulement les services de base (recommandé pour le développement)
make up

# Vérifier que tout fonctionne
make health

# Accéder à l'application

```

```
open http://localhost:8080 # Frontend
open http://localhost:3000/health # API Health
```

Tous les services

```
# Démarrer tous les services (15+ services)
make up-full

# Ou démarrer par profils
make up-profile PROFILE=auth # Authentification
make up-profile PROFILE=applications # Candidatures
make up-profile PROFILE=companies # Entreprises
make up-profile PROFILE=monitoring # Monitoring complet
```

Mode développement avec rechargement automatique

```
# Mode développement avec hot reload
make dev

# Ou pour un service spécifique
make dev-frontend
make dev-backend
```

Développement backend

Structure d'un service

Chaque service suit une architecture standardisée :

```
// backend/auth-service/src/server.js
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const { PrismaClient } = require('@prisma/client');

const authRoutes = require('./routes/auth');
const { errorHandler } = require('./middlewares/errorHandler');

const app = express();
const prisma = new PrismaClient();

// Middleware de sécurité
```

```

app.use(helmet());
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') || ['http://localhost:8080'],
  credentials: true
}));
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

// Routes
app.use('/auth', authRoutes);

// Health check
app.get('/health', (req, res) => {
  res.json({
    status: 'healthy',
    service: 'auth-service',
    timestamp: new Date().toISOString(),
    uptime: process.uptime()
  });
});

// Gestion des erreurs
app.use(errorHandler);

// Démarrage du serveur
const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
  console.log(`Auth Service running on port ${PORT}`);
});

module.exports = app;

```

Contrôleurs API

```

// backend/auth-service/src/controllers/authController.js
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();

class AuthController {
  async login(req, res) {
    try {
      const { email, password } = req.body;

      // Validation
      if (!email || !password) {
        return res.status(400).json({

```

```

        success: false,
        error: {
            code: 'VALIDATION_ERROR',
            message: 'Email et mot de passe requis'
        }
    });
}

// Rechercher l'utilisateur
const user = await prisma.user.findUnique({
    where: { email }
});

if (!user) {
    return res.status(401).json({
        success: false,
        error: {
            code: 'AUTHENTICATION_ERROR',
            message: 'Identifiants invalides'
        }
    });
}

// Vérifier le mot de passe
const validPassword = await bcrypt.compare(password, user.passwordHash);
if (!validPassword) {
    return res.status(401).json({
        success: false,
        error: {
            code: 'AUTHENTICATION_ERROR',
            message: 'Identifiants invalides'
        }
    });
}

// Générer les tokens
const accessToken = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '1h' }
);

const refreshToken = jwt.sign(
    { userId: user.id },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: process.env.JWT_REFRESH_EXPIRES_IN || '7d' }
);

// Mettre à jour la dernière connexion
await prisma.user.update({
    where: { id: user.id },
    data: { lastLoginAt: new Date() }
});

```

```

    });

    res.json({
      success: true,
      data: {
        accessToken,
        refreshToken,
        user: {
          id: user.id,
          email: user.email,
          name: user.name,
          role: user.role
        }
      }
    });

  } catch (error) {
    console.error('Login error:', error);
    res.status(500).json({
      success: false,
      error: {
        code: 'INTERNAL_ERROR',
        message: 'Erreur interne du serveur'
      }
    });
  }
}
}

module.exports = new AuthController();

```

Services métier

```

// backend/auth-service/src/services/authService.js
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();

class AuthService {
  async createUser(userData) {
    const { email, password, name, role = 'user' } = userData;

    // Vérifier si l'utilisateur existe déjà
    const existingUser = await prisma.user.findUnique({
      where: { email }
    });
  }
}

```

```

if (existingUser) {
  throw new Error('User already exists');
}

// Hasher le mot de passe
const passwordHash = await bcrypt.hash(password, 12);

// Créer l'utilisateur
const user = await prisma.user.create({
  data: {
    email,
    passwordHash,
    name,
    role
  }
});

return user;
}

async generateTokens(userId) {
  const accessToken = jwt.sign(
    { userId },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );

  const refreshToken = jwt.sign(
    { userId },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: '7d' }
  );

  return { accessToken, refreshToken };
}
}

module.exports = new AuthService();

```

Développement frontend

Structure Next.js

```

frontend/
├─ src/
│   └─ app/                # App Router (Next.js 14+)
│       └─ (auth)/          # Routes d'authentification
│           └─ (dashboard)/ # Routes du dashboard

```

```

|   |   └─ api/           # API Routes Next.js
|   |   └─ layout.tsx     # Layout principal
|   └─ components/        # Composants React
|   |   └─ ui/            # Composants UI de base
|   |   └─ forms/         # Composants de formulaires
|   |   └─ charts/        # Composants de graphiques
|   └─ hooks/             # Hooks personnalisés
|   └─ lib/               # Utilitaires et configurations
|   |   └─ api.ts         # Client API
|   |   └─ auth.ts        # Gestion de l'authentification
|   |   └─ utils.ts       # Utilitaires
|   └─ types/             # Types TypeScript
|   └─ styles/            # Styles et CSS
└─ public/               # Assets statiques
└─ middleware.ts         # Middleware Next.js
└─ next.config.js        # Configuration Next.js

```

Client API TypeScript

```

// frontend/src/lib/api.ts
interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: {
    code: string;
    message: string;
    details?: any[];
  };
  timestamp: string;
}

class ApiClient {
  private baseUrl: string;
  private token: string | null = null;

  constructor(baseUrl: string = 'http://localhost:3000') {
    this.baseUrl = baseUrl;
    this.token = typeof window !== 'undefined' ? localStorage.getItem('auth_token')
  }

  setToken(token: string) {
    this.token = token;
    if (typeof window !== 'undefined') {
      localStorage.setItem('auth_token', token);
    }
  }
}

```

```

clearToken() {
  this.token = null;
  if (typeof window !== 'undefined') {
    localStorage.removeItem('auth_token');
  }
}

private async request<T>(  
  endpoint: string,  
  options: RequestInit = {}  
) : Promise<ApiResponse<T>> {  
  const url = `${this.baseURL}${endpoint}`;  
  
  const config: RequestInit = {  
    headers: {  
      'Content-Type': 'application/json',  
      ...(this.token && { Authorization: `Bearer ${this.token}` })),  
      ...options.headers,  
    },  
    ...options,  
  };  
  
  try {  
    const response = await fetch(url, config);  
  
    if (!response.ok) {  
      const error = await response.json();  
      throw new Error(error.error?.message || `HTTP ${response.status}`);  
    }  
  
    return await response.json();  
  } catch (error) {  
    console.error('API Request failed:', error);  
    throw error;  
  }  
}  
  
// Authentication  
async login(email: string, password: string) {  
  const response = await this.request<{ accessToken: string; user: any }>('/auth.  
    method: 'POST',  
    body: JSON.stringify({ email, password })),  
  );  
  
  if (response.success && response.data) {  
    this.setToken(response.data.accessToken);  
  }  
  
  return response;  
}  
  
async logout() {

```



```

    await this.request('/auth/logout', { method: 'POST' });
    this.clearToken();
  }

  // Candidatures
  async getApplications(filters?: any) {
    const params = new URLSearchParams(filters);
    return this.request(`/applications?${params}`);
  }

  async createApplication(data: any) {
    return this.request('/applications', {
      method: 'POST',
      body: JSON.stringify(data),
    });
  }

  // Entreprises
  async getCompanies() {
    return this.request('/companies');
  }

  async createCompany(data: any) {
    return this.request('/companies', {
      method: 'POST',
      body: JSON.stringify(data),
    });
  }
}

export const apiClient = new ApiClient();

```

Développement mobile (Flutter)

Structure de l'application

```

flutter-mobile-app/
├─ lib/
│   ├─ models/           # Modèles de données
│   ├─ services/         # Services API et locaux
│   ├─ providers/        # State management (Provider)
│   ├─ screens/          # Écrans de l'application
│   ├─ widgets/          # Composants réutilisables
│   ├─ utils/            # Utilitaires
│   └─ main.dart          # Point d'entrée
└─ test/                 # Tests unitaires

```

```
└─ integration_test/      # Tests d'intégration
└─ pubspec.yaml           # Dépendances Flutter
```

Service API Flutter

```
// lib/services/api_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:flutter/foundation.dart';

class ApiService {
  static const String baseUrl = 'http://localhost:3000';
  String? _token;

  void setToken(String token) {
    _token = token;
  }

  void clearToken() {
    _token = null;
  }

  Map<String, String> get _headers => {
    'Content-Type': 'application/json',
    if (_token != null) 'Authorization': 'Bearer $_token',
  };

  Future<ApiResponse<T>> request<T>(
    String endpoint, {
    String method = 'GET',
    Map<String, dynamic>? data,
  }) async {
    try {
      final url = Uri.parse('$baseUrl$endpoint');
      final body = data != null ? jsonEncode(data) : null;

      final response = await http.request(
        url,
        method: method,
        headers: _headers,
        body: body,
      );

      if (response.statusCode >= 200 && response.statusCode < 300) {
        final jsonResponse = jsonDecode(response.body);
        return ApiResponse<T>.fromJson(jsonResponse);
      } else {
        throw Exception('HTTP ${response.statusCode}: ${response.body}');
      }
    }
  }
}
```

```

    } catch (e) {
        debugPrint('API Error: $e');
        rethrow;
    }
}

// Authentication
Future<ApiResponse<LoginResponse>> login(String email, String password) async {
    final response = await request<LoginResponse>(
        '/auth/login',
        method: 'POST',
        data: {'email': email, 'password': password},
    );

    if (response.success && response.data != null) {
        setToken(response.data!.accessToken);
    }

    return response;
}

// Candidatures
Future<ApiResponse<List<Application>>> getApplications() async {
    return request<List<Application>>('/applications');
}

Future<ApiResponse<Application>> createApplication(Map<String, dynamic> data) async {
    return request<Application>('/applications', method: 'POST', data: data);
}

// Entreprises
Future<ApiResponse<List<Company>>> getCompanies() async {
    return request<List<Company>>('/companies');
}
}

class ApiResponse<T> {
    final bool success;
    final T? data;
    final ApiError? error;
    final String timestamp;

    ApiResponse({
        required this.success,
        this.data,
        this.error,
        required this.timestamp,
    });

    factory ApiResponse.fromJson(Map<String, dynamic> json) {
        return ApiResponse(
            success: json['success'],

```

```

        data: json['data'],
        error: json['error'] != null ? ApiError.fromJson(json['error']) : null,
        timestamp: json['timestamp'],
    );
}

class ApiError {
    final String code;
    final String message;
    final List<dynamic>? details;

    ApiError({
        required this.code,
        required this.message,
        this.details,
    });

    factory ApiError.fromJson(Map<String, dynamic> json) {
        return ApiError(
            code: json['code'],
            message: json['message'],
            details: json['details'],
        );
    }
}

```

Tests et qualité

Stratégie de tests

Tests unitaires

- **Backend** : Tests des contrôleurs, services, utilitaires
- **Frontend** : Tests des composants React, hooks, utilitaires
- **Mobile** : Tests des widgets Flutter, services, modèles

Tests d'intégration

- Tests des interactions entre services
- Tests des API endpoints
- Tests de la base de données

Tests end-to-end (E2E)

- Tests complets des workflows utilisateurs
- Tests de l'interface web et mobile

- Tests de performance

Configuration des tests

Backend (Jest)

```
// backend/auth-service/jest.config.js
module.exports = {
  testEnvironment: 'node',
  roots: ['<rootDir>/src', '<rootDir>/tests'],
  testMatch: ['**/__tests__/**/*.js', '**/?(*.)+(spec|test).js'],
  collectCoverageFrom: [
    'src/**/*.js',
    '!src/server.js',
    '!src/**/*.test.js',
    '!src/**/*.__tests__/**'
  ],
  coverageDirectory: 'coverage',
  coverageReporters: ['text', 'lcov', 'html'],
  setupFilesAfterEnv: ['<rootDir>/tests/setup.js'],
  testTimeout: 10000,
};
```

Frontend (Jest + React Testing Library)

```
// frontend/jest.config.js
module.exports = {
  testEnvironment: 'jsdom',
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],
  moduleNameMapping: {
    '^@/(.*)$': '<rootDir>/src/$1',
  },
  testMatch: [
    '<rootDir>/src/**/*.__tests__/**/*.{js,jsx,ts,tsx}',
    '<rootDir>/src/**/*.{test,spec}.{js,jsx,ts,tsx}'
  ],
  collectCoverageFrom: [
    'src/**/*.{js,jsx,ts,tsx}',
    '!src/**/*.d.ts',
    '!src/**/*.__tests__/**',
    '!src/**/*.test.{js,jsx,ts,tsx}',
    '!src/**/*.spec.{js,jsx,ts,tsx}'
  ],
  coverageDirectory: 'coverage',
};
```

```
coverageReporters: ['text', 'lcov', 'html'],
};
```

Tests E2E (Playwright)

```
// frontend/playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests/e2e',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
  ],
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

Exécution des tests

```
# Tests backend
make test-backend          # Tests de tous les services backend
make test-backend-unit     # Tests unitaires backend
make test-backend-integration # Tests d'intégration backend

# Tests frontend
make test-frontend         # Tests frontend
make test-frontend-unit    # Tests unitaires frontend
make test-frontend-e2e     # Tests E2E frontend

# Tests mobile
```

```
make test-mobile           # Tests Flutter
make test-mobile-unit      # Tests unitaires mobile
make test-mobile-integration # Tests d'intégration mobile

# Tests complets
make test                  # Tous les tests
make test-coverage         # Tests avec couverture de code
make test-ci               # Tests pour CI/CD

# Tests spécifiques
make test-service SERVICE=auth-service
make test-api-endpoints    # Tests des endpoints API
```

Qualité du code

Linting et formatage

```
# Backend
make lint-backend          # ESLint backend
make format-backend        # Prettier backend

# Frontend
make lint-frontend         # ESLint frontend
make format-frontend       # Prettier frontend

# Mobile
make lint-mobile           # Dart analyze
make format-mobile         # Dart format

# Tous
make lint                  # Linting complet
make format                # Formatage complet
```

Analyse statique

```
# Analyse de sécurité
make security-scan         # Scan de sécurité
make dependency-check      # Vérification des dépendances

# Analyse de performance
make performance-check     # Analyse des performances
make bundle-analyzer       # Analyse des bundles
```

Outils de développement

Commandes Make principales

Démarrage et arrêt

```
make help           # Afficher l'aide complète
make up             # Services essentiels (postgres, redis, api-gateway)
make up-full        # Tous les services
make down           # Arrêter tous les services
make restart        # Redémarrer tous les services
```

Développement

```
make dev            # Mode développement avec hot reload
make dev-frontend  # Développement frontend uniquement
make dev-backend   # Développement backend uniquement
make logs           # Logs de tous les services
make logs-service SERVICE=auth # Logs d'un service spécifique
```

Base de données

```
make db-migrate     # Appliquer les migrations
make db-seed        # Insérer les données de test
make db-reset       # Reset de la base de données
make db-backup      # Backup de la base
make db-studio      # Prisma Studio (interface web)
```

Tests et qualité

```
make test           # Tous les tests
make test-backend   # Tests backend
make test-frontend  # Tests frontend
make test-mobile    # Tests mobile
make test-coverage  # Tests avec couverture
make lint           # Linting complet
make format         # Formatage du code
```


Docker

```
make build           # Construire toutes les images
make clean           # Nettoyer les images et volumes
make ps              # État des conteneurs
make docker-exec SERVICE=postgres # Accès shell dans un conteneur
```

Scripts utilitaires

Scripts de setup

```
# Installation complète
./scripts/setup/install.sh

# Configuration des services
./scripts/setup/configure-services.sh

# Initialisation de la base de données
./scripts/database/init.sh
```

Scripts de développement

```
# Monitoring de la santé
./scripts/health/check-all.sh

# Backup automatique
./scripts/database/backup.sh

# Nettoyage des logs
./scripts/docker/clean-logs.sh
```

Scripts de déploiement

```
# Déploiement en production
./scripts/deploy/production.sh

# Tests de déploiement
./scripts/deploy/test-deployment.sh
```

```
# Rollback
./scripts/deploy/rollback.sh
```

IDE et extensions

Visual Studio Code

Extensions recommandées :

```
{
  "recommendations": [
    "ms-vscode.vscode-docker",
    "ms-vscode.vscode-nodejs",
    "ms-vscode.vscode-typescript-next",
    "esbenp.prettier-vscode",
    "dbaeumer.vscode-eslint",
    "ms-vscode.vscode-json",
    "ms-vscode.vscode-markdown",
    "ms-vscode.vscode-git-graph",
    "ms-vscode.vscode-prisma",
    "dart-code.flutter",
    "ms-vscode.vscode-yaml",
    "redhat.vscode-yaml",
    "ms-vscode.vscode-env"
  ]
}
```

Configuration debugging

```
// .vscode/launch.json
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug Frontend",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/frontend/node_modules/next/dist/bin/next",
      "args": ["dev"],
      "cwd": "${workspaceFolder}/frontend",
      "console": "integratedTerminal"
    },
    {
      "name": "Debug Auth Service",
      "type": "node",

```

```

    "request": "launch",
    "program": "${workspaceFolder}/backend/auth-service/src/server.js",
    "cwd": "${workspaceFolder}/backend/auth-service",
    "console": "integratedTerminal"
  },
  {
    "name": "Debug API Gateway",
    "type": "node",
    "request": "launch",
    "program": "${workspaceFolder}/backend/api-gateway/src/server.js",
    "cwd": "${workspaceFolder}/backend/api-gateway",
    "console": "integratedTerminal"
  }
]
}

```



Monitoring et debugging

Monitoring en temps réel

Services de monitoring

```

# Interface web Metrics Aggregator
open http://localhost:3014

# cAdvisor (métriques Docker)
open http://localhost:8081

# Prometheus (métriques système)
open http://localhost:9090

# API Health
curl http://localhost:3000/health

```

Logs en temps réel

```

# Tous les logs
make logs

# Logs d'un service spécifique
make logs-auth
make logs-frontend
make logs-postgres

```

```
# Logs avec filtrage
make logs | grep ERROR
make logs | grep "api-gateway"
```

Debugging

Debug d'un service

```
# Accès shell dans un conteneur
make docker-exec SERVICE=postgres
make docker-exec SERVICE=auth-service
make docker-exec SERVICE=frontend

# Debug avec variables d'environnement
DEBUG=auth:* make logs-auth
DEBUG=app:* make logs-frontend
```

Outils de debugging

```
# Vérifier les ports utilisés
make check-ports

# Vérifier l'état des services
make status

# Health check détaillé
make health-detailed

# Monitoring des ressources
make docker-stats
```

Profiling

```
# Profilage Node.js
NODE_OPTIONS="--prof" make dev-backend

# Analyse du profiling
node --prof-process isolate-*.log
```

Déploiement

Déploiement local

Développement

```
# Services de base
make up

# Tous les services
make up-full

# Avec monitoring
make up-profile PROFILE=monitoring
```

Tests

```
# Déploiement de test
make up-profile PROFILE=test

# Tests automatisés
make test-e2e

# Tests de performance
make performance-test
```

Déploiement en production

Préparation

```
# Build des images de production
make build-production

# Tests de production
make test-production

# Configuration production
cp .env.production .env
```

Déploiement

```
# Déploiement avec Docker Compose
make deploy-production

# Ou avec Docker Swarm
make deploy-swarm

# Ou avec Kubernetes
make deploy-kubernetes
```

Vérification

```
# Health check production
make health-production

# Logs de production
make logs-production

# Monitoring production
open https://monitoring.jobbingtrack.com
```

Consultez le [guide de déploiement en production](#) pour plus de détails.



Contribution

Workflow Git

1. Préparation

```
# Fork le repository
git clone https://github.com/votre-username/JobbingTrack.git
cd JobbingTrack

# Configuration Git
git config user.name "Votre Nom"
git config user.email "votre.email@example.com"
```

2. Création de branche

```
# Créer une branche feature
git checkout -b feature/nouvelle-fonctionnalite

# Ou une branche bugfix
git checkout -b bugfix/correction-bug

# Ou une branche hotfix
git checkout -b hotfix/correction-critique
```

3. Développement

```
# Installer les dépendances
make install

# Démarrer l'environnement
make up

# Développer et tester
make test

# Formatage du code
make format
make lint
```

4. Commit et push

```
# Ajouter les fichiers
git add .

# Commit avec message conventionnel
git commit -m "feat: ajouter authentication JWT"

# Push vers la branche
git push origin feature/nouvelle-fonctionnalite
```

Standards de code

Conventions de commit

```
# Types de commit
feat:      # Nouvelle fonctionnalité
```

```
fix:      # Correction de bug
docs:     # Documentation
style:    # Formatage, point-virgule manquante
refactor: # Refactorisation
test:     # Tests
chore:    # Maintenance
```

Exemples de messages

```
feat: ajouter service de notifications
fix:  corriger la validation des emails
docs: mettre à jour la documentation API
style: formater le code avec Prettier
refactor: extraire les utilitaires de validation
test:  ajouter tests unitaires pour l'authentification
chore: mettre à jour les dépendances
```

Pull Request

Template de PR

Description

Description claire et concise de la modification

Type de changement

- [] Bug fix (correction non-breaking)
- [] New feature (ajout non-breaking)
- [] Breaking change (modification breaking)

Tests

- [] Tests unitaires ajoutés/modifiés
- [] Tests d'intégration ajoutés/modifiés
- [] Tests E2E ajoutés/modifiés

Documentation

- [] Documentation mise à jour
- [] Guide utilisateur mis à jour
- [] API documentation mise à jour

Checklist avant merge

- [] Code review effectuée
- [] Tests passent
- [] Linting OK

- [] Formatage OK
- [] Conflits résolus

Process de review

1. **Assignment** : Assigner 2 reviewers minimum
2. **Tests** : Tous les tests doivent passer
3. **Code review** : Review approfondie du code
4. **Documentation** : Vérifier la documentation
5. **Merge** : Merge après approval



Résolution de problèmes

Problèmes courants

1. Services qui ne démarrent pas

```
# Solution complète
make down
make clean
make up

# Ou redémarrage sélectif
make restart-service SERVICE=api-gateway
make restart-service SERVICE=postgres
```

2. Problèmes de base de données

```
# Reset de la base
make db-reset

# Recréer les schémas
make db-migrate

# Réinsérer les données
make db-seed

# Accès direct PostgreSQL
make docker-exec SERVICE=postgres
# Puis dans le conteneur :
psql -U jobbingtrack -d jobbingtrack
```

3. Problèmes de ports

```
# Vérifier les ports utilisés
make check-ports

# Ports couramment utilisés :
# 3000: API Gateway
# 3001: Auth Service
# 3002: Application Service
# 3003: Company Service
# 5432: PostgreSQL
# 6379: Redis
# 8080: Frontend
# 8081: cAdvisor

# Solution : modifier les ports dans docker-compose.yml
```

4. Problèmes de mémoire

```
# Vérifier l'utilisation mémoire
make docker-stats

# Nettoyer les conteneurs arrêtés
make clean-containers

# Redémarrer Docker
sudo systemctl restart docker
```

5. Problèmes de réseau

```
# Vérifier la connectivité
make health

# Tester la connectivité entre services
make docker-exec SERVICE=api-gateway
# Dans le conteneur :
curl http://auth-service:3001/health
curl http://postgres:5432
```

Debugging avancé

Logs détaillés

```
# Logs avec timestamps
make logs-detailed

# Logs JSON formatés
make logs-json

# Logs d'erreurs uniquement
make logs | grep -i error
make logs | grep -E "(ERROR|Exception|Failed)"
```

Debug interactif

```
# Attacher un debugger Node.js
make debug-attach SERVICE=auth-service

# Debug avec Chrome DevTools
make debug-chrome

# Profiling mémoire
make memory-profile
```

Monitoring des performances

```
# Métriques temps réel
watch -n 5 'curl -s http://localhost:3000/health'

# Monitoring des ressources
make monitor-resources

# Alertes de performance
make performance-alerts
```

Base de données

```
# Analyser les requêtes lentes
make db-slow-queries

# Statistiques PostgreSQL
make db-stats

# Backup et restauration
```

```
make db-backup
make db-restore BACKUP_FILE=backup_2025-01-27.sql
```

Ressources supplémentaires

Ressource	Description	Lien
Architecture	Vue d'ensemble technique	ARCHITECTURE.md
API	Documentation des endpoints	API.md
Services	Détail des microservices	SERVICES.md
Makefile	Commandes automatisées	MAKEFILE.md
Tests	Stratégies de test	TESTING-GUIDE.md
Dépannage	Résolution des problèmes	TROUBLESHOOTING.md
Déploiement	Guide de déploiement	deployment-guide.md
Sécurité	Bonnes pratiques	security-guide.md
Scripts	Automatisations	scripts/README.md

Navigation

-  [Index Documentation](#)
-  [README Principal](#)
-  [Architecture](#)
-  [API](#)
-  [Services](#)
-  [Tests](#)
-  [Dépannage](#)

Dernière mise à jour: Octobre 2025 **Version:** 4.0 - Guide de développement complet **Environnement:** Docker + Node.js + PostgreSQL + Redis **Tests:** Jest + Playwright + Supertest **Outils:** Make + ESLint + Prettier